

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# SE 423: Introduction to Mechatronics

## Lecture 16: LiDAR Code

Marius Juston

Monday, March 23rd, 2026

# Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- Why did we use a ping pong buffer for the FFT code?
- Given that LiDAR splits its space as 360 degrees into 1024 steps, what is the angular resolution of the LiDAR?
- What happens if your angular resolution is too low and your object is far away?
- Is UART a synchronous or asynchronous communication protocol? What does it mean for it to be synchronous or not?

# Reminders

**Office Hours:**

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

**Lab:** Starting Lab 6 this week. This a 3-week lab!

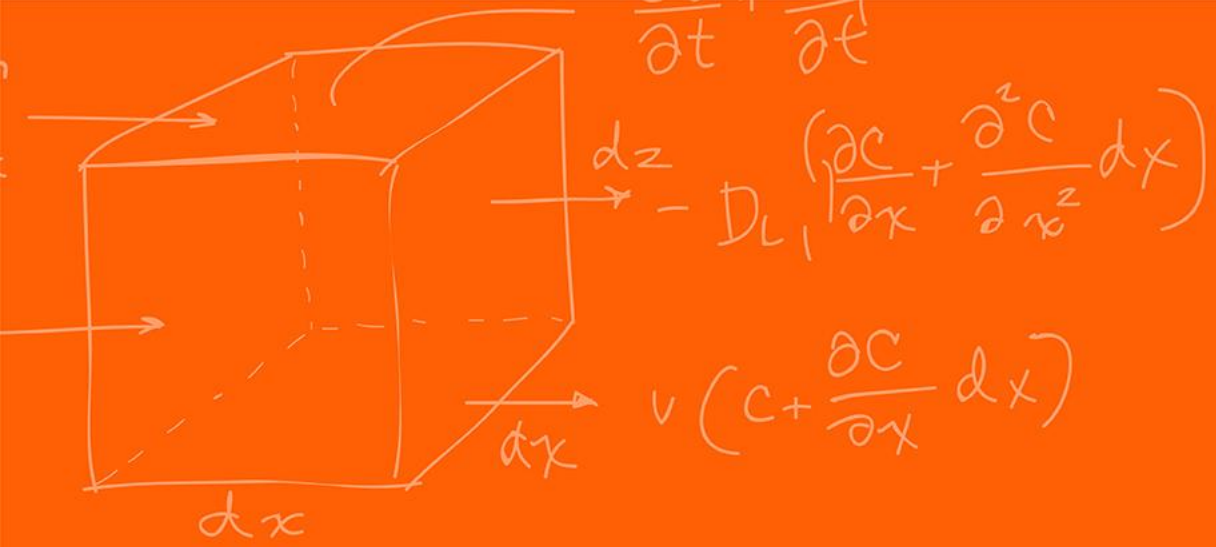
- Don’t forget to have the Lab LabVIEW application implemented before lab.

**Homeworks:**

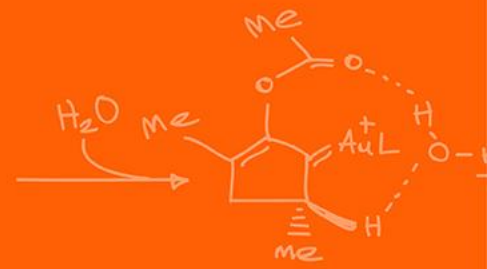
- Check-offs for homework 3 Ex 5 are due **March 24, 5PM (The end of office hours)**
  - This is a long homework, so **DO NOT** expect to work on and finish it during the Tuesday office hours time!
- Answers and code submissions for homework 3 are due **March 25, 9AM**
- Check-offs for homework 4 Ex 2 and 3 are due **March 31, 5PM (The end of office hours)**

# Questions?

Homework, Lab, LabVIEW?

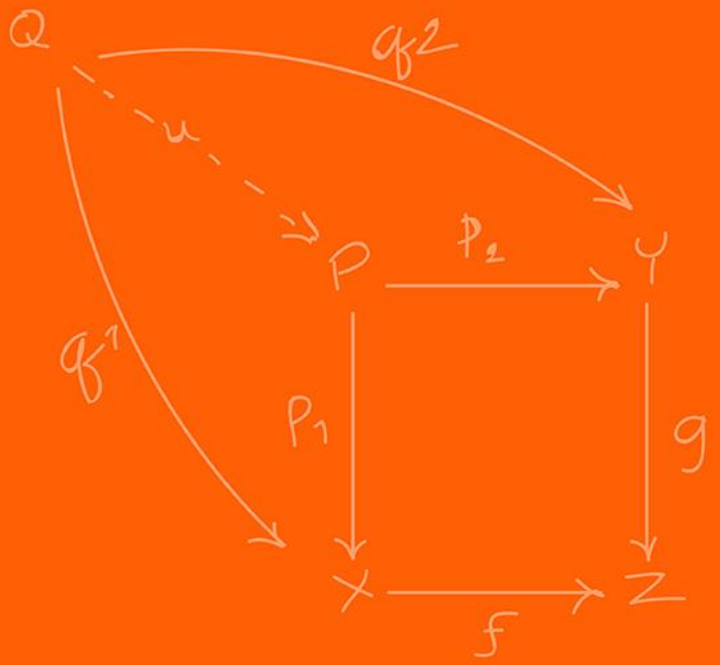






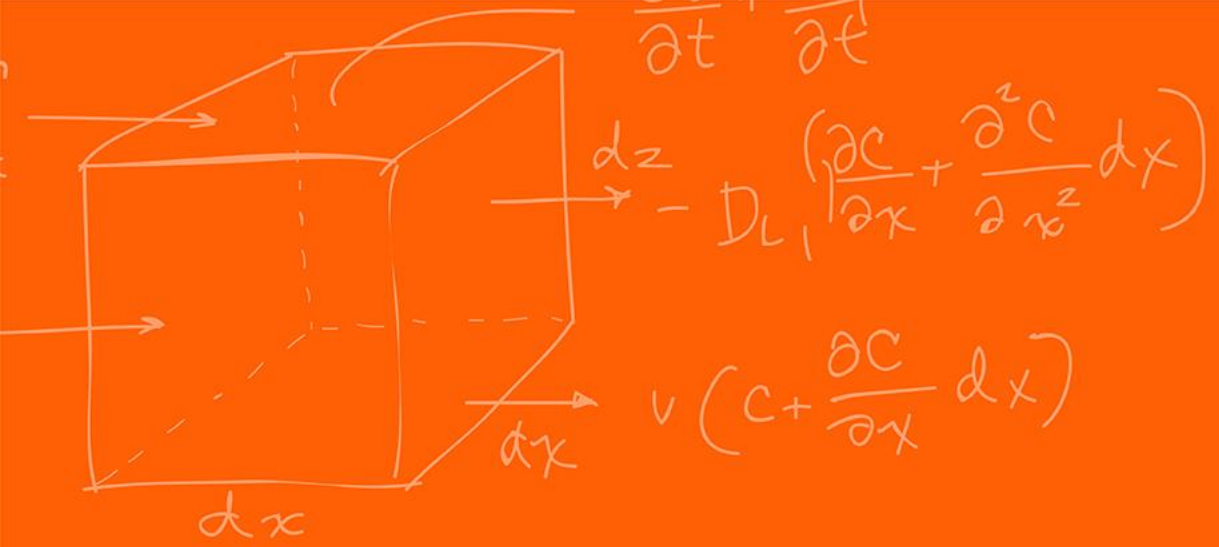


# URG-04LX

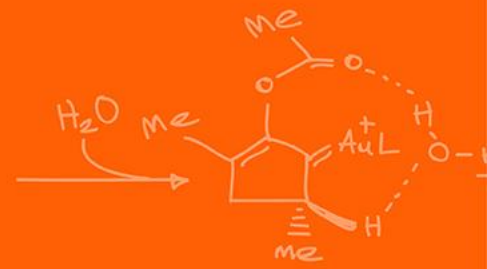


# LiDAR

- The LiDAR we are working with is called the URG-04LX
- RG-04LX is a laser sensor for area scanning. The light source of the sensor is infrared laser of wavelength 785nm with laser class 1 safety.

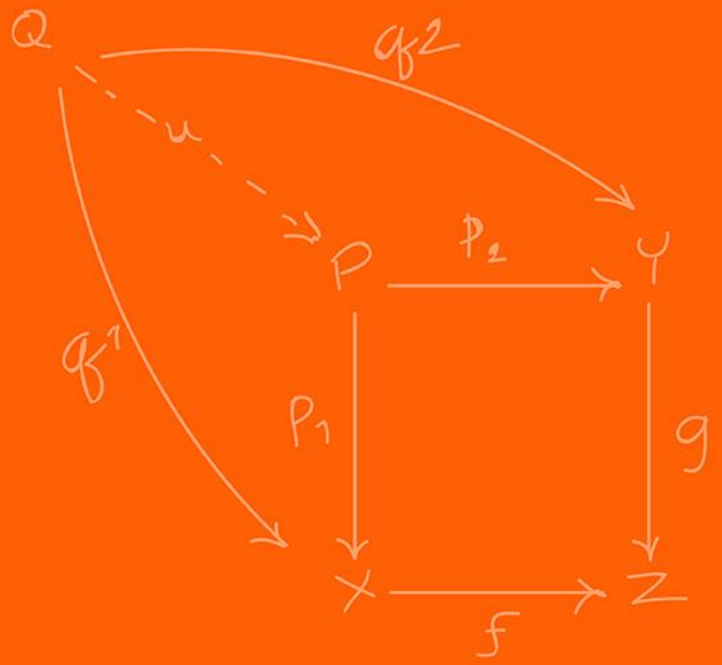








# Laser Classes



# LiDAR – Laser classes

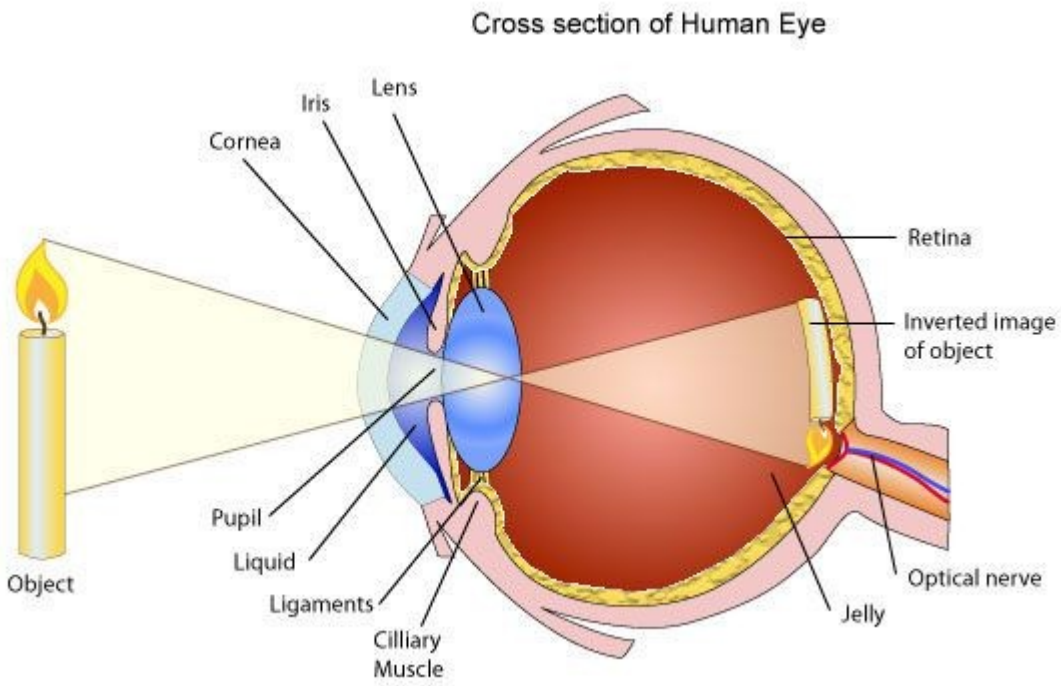
- Laser radiation can cause three basic types of laser hazards: **eye**, **skin** and **fire hazards**.

- If a laser is not class 1 compliant, you should wear protective equipment when entering the danger zone: laser safety glasses for eye protection and special suits for skin protection.

| Lazer Class | Hazard Risk | Common Uses | Safe Use Considerations |
|---|---|---|---|
| **Class 1** | None | Barcode scanners, DVD players (if you remember what a DVD is!) | Safe under all considerations. |
| **Class 1M** | None to low | Laser diodes, fiber communication systems, speed meters | Safe unless viewed with optical instruments |
| **Class 2** | Low (unless stared into) | Laser points, alignment tools | Avoid direct eye exposure |
| **Class 2M** | Low to moderate | Projection laser (e.g. light shows) | Safe unless viewed with optical instruments |
| **Class 3R/3B** | Moderate to high | Laser scanners, 3D printers | Can damage the naked eye and skin |
| **Class 4** | Very high | Cutting, marking welding, cleaning surgery | Requires protective eyewear and strict controls. |

# LiDAR – Laser classes – Eye

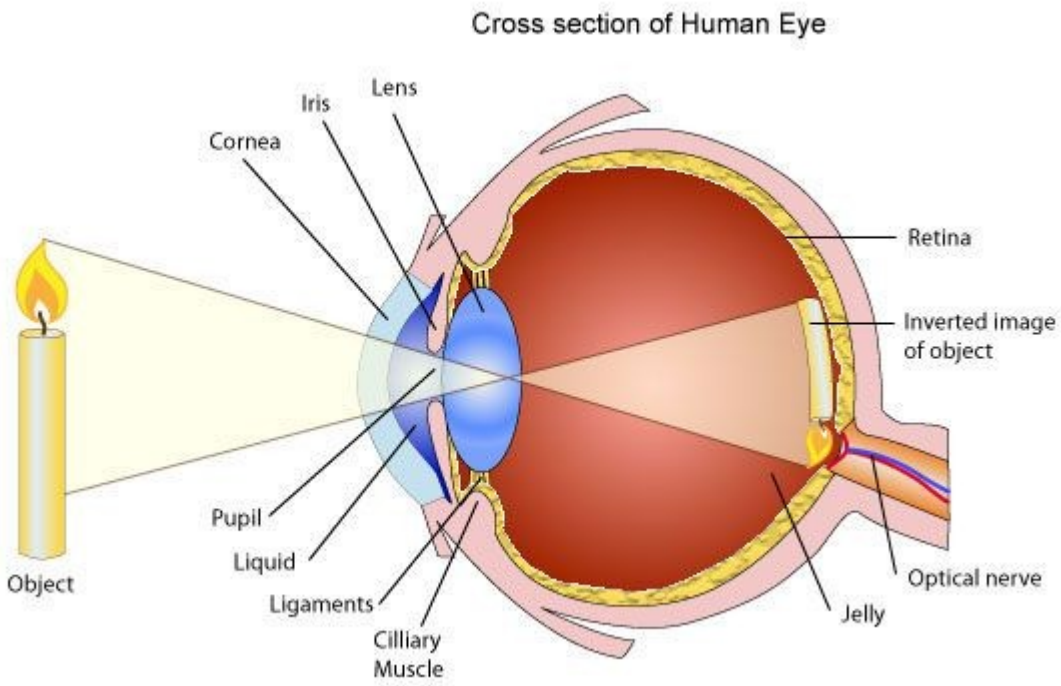
- Of all the laser hazards, eye injuries are the most serious (unless you don't mind being blind!).
- When light reaches the eye, the **cornea** and the **lens** act as **amplifiers**. They concentrate light onto the **retina** (the back of the eye) which, afterward, is processed by the brain as an image.
- Those three components of the human eye (**cornea, lens and retina**) are the most susceptible to damage from laser radiation.
- Almost all types of laser lights can harm your eyes, but the various components of the human eye react more strongly with some light wavelengths.




https://www.laserax.com/blog/laser-safety-laser-classes-explained

# LiDAR – Laser classes – Eye

- Part of visible light is absorbed by the eyes before being amplified by the lens and cornea.
- But infrared light isn't visible and isn't absorbed by the eyes. Thus, is more dangerous than visible light.
- All this energy is spent burning a small area of the retina which causes blindness and severe ey damages. Photochemical damage is also possible for lights lower than 400 nm (in the ultraviolet range) and may cause cataracts (decreased vision).
- Protective eyewear like laser goggles protects you by absorbing dangerous light.
- **Different goggles absorb different lights**, so you need to wear the right goggles for your laser class.




https://www.laserax.com/blog/laser-safety-laser-classes-explained

# LiDAR – Laser classes – Skin

- If you had the choice between putting your eyes or your hands on the stove, chances are you would put your hands. It’s easy to understand why skin hazards are less serious than eye hazards. But the risks of skin burns still deserve attention.

- Direct contact with the laser beam and specular (mirror-like) reflections can cause skin injuries. Those injuries are typically caused by thermal damage like touching the stove, or photochemical damage like sunburns.

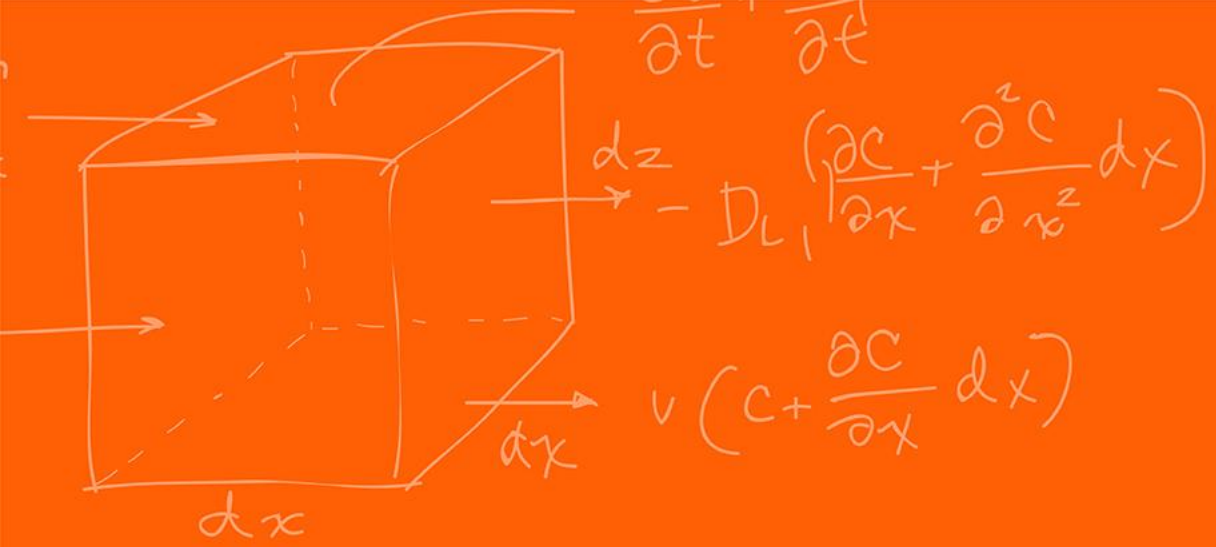
- The burn level depends on the laser’s output power, the wavelength, the size of the affected area, and the duration of the irradiation.

# LiDAR – Laser classes – Fire hazards

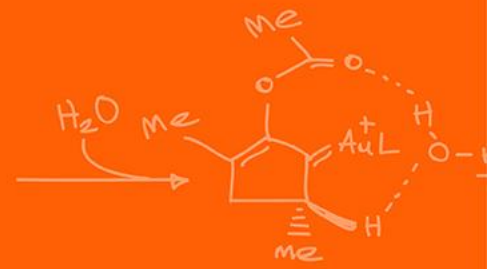
- Apart from health hazards, laser light can also start fires and put your work environment at risk.
- Only class 4 lasers pose real fire safety concerns. Their direct beam as well as any type of reflection can ignite combustible materials.
- For a safe integration, those lasers must be enclosed properly and account for all possible angles of reflection, including diffuse reflections.

Given that the URG-04LX is a class 1 laser. What precautions do we need to take when handling it?

Nothing, it is a class 1 laser!

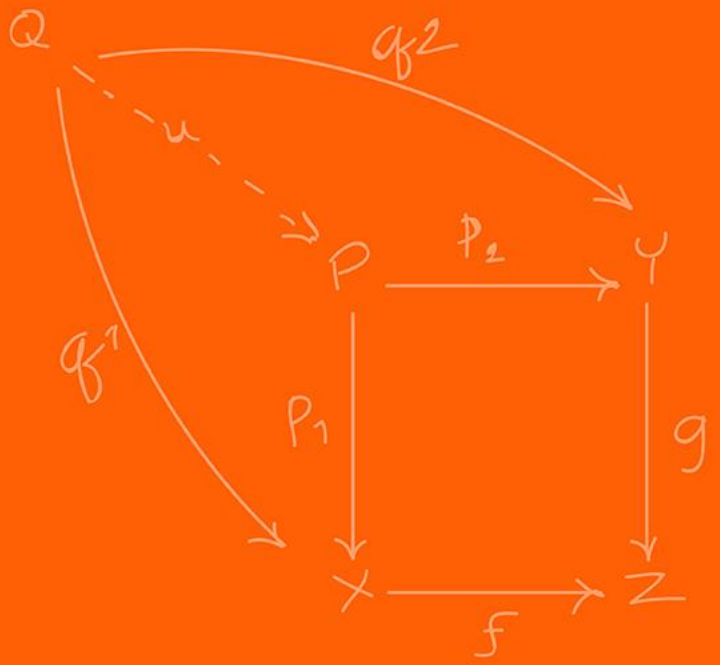








# URG-04LX



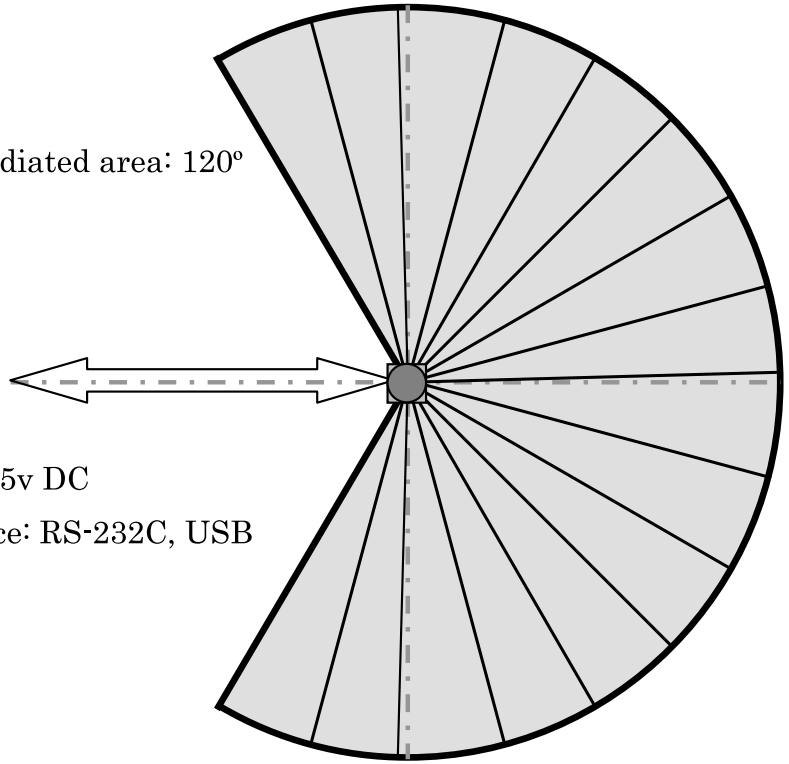
# LiDAR

- Scan area is 240° semicircle with maximum distance of 4000mm.
- Pitch angle is 0.36° and sensor outputs the distance measured at every point (683 steps).
- Laser beam diameter is less than 20mm at 2000mm with maximum divergence 40mm at 4000mm.
- The principle of distance measurement is based on calculation of the phase difference, due to which it is possible to obtain stable measurement with minimum influence from object's color and surface gloss.
  - Essentially Time of Flight but with a continuous wave rather than the discrete pulse

# LiDAR




https://www.robotshop.com/products/hokuyo-urg-04lx-laser-rangefinder



Non-radiated area: 120º

Power: 5v DC

Interface: RS-232C, USB

Detection Area: 240º

Max. Distance: 4000mm

# LiDAR

- There are 2 ways to communicate with the LiDAR sensor SCI (UART) or USB. We use SCI only.

- There are 4 types of commands that can be sent.

1. Version Command (‘V’)
2. Laser Illumination Command (‘L’)
3. Communication Speed Setting command for RS-232C (‘S’)
4. Distance Data Acquisition Command (‘G’)

# LiDAR

- The SCI data packets are in the following format,

Host to Sensor:

| Command | Parameter (if necessary) | LF ($0a_{16}$) or CR ($0d_{16}$) |
|---|---|---|

Sensor to Host:

| Command | Parameter (if necessary) | LF | Status | LF | Data | LF | LF |
|---|---|---|---|---|---|---|---|

- Termination code is either Line Feed (‘\n’) or Carriage Return (‘\r’)
- The codes $XX_{16}$ represents ASCII codes in hex
- Any status code other than $‘0’_{16}$ represents an error.

# LiDAR – Version Info

- If we want to get the version information of the LiDAR we can use the version command. This can be done simply as sending,
- The command ‘V’

(HOST->SENSOR)

| | |
|---|---|
| ‘V’($56_{16}$) | LF or CR |

(SENSOR->HOST)

| | | |
|---|---|---|
| ‘V’($56_{16}$) | LF | |
| Status | LF | |
| Vender Information | LF | |
| Product Information | LF | |
| Firmware Version | LF | |
| Protocol Version | LF | |
| Sensor Serial Number | LF | LF |

Example :

V[LF]
0[LF]
VEND:Hokuyo Automatic Co., Ltd.[LF]
PROD: SOKUIKI Sensor URG-X002[LF]
FIRM:0.01.22a,(2004/12/27)[LF]
PROT:00001,(SCIP 1.0)[LF]
SERI: H0400000[LF][LF]

- Done as:

```
serial_sendSCIC(&SerialC, "V\n", 2);
```

# LiDAR – Laser Illumination Control

- If we want to turn the LiDAR on or off, we can use the ‘L’ command to turn it on or off.
- By default, the sensor is switched on even if it was previously switched off

(HOST->SENSOR)

| | | |
|---|---|---|
| ‘L’($4C_{16}$) | Control Code (1 Byte) | LF or CR |

(SENSOR->HOST)

| | | | | | |
|---|---|---|---|---|---|
| ‘L’($4C_{16}$) | Control Code (1 Byte) | LF | Status | LF | LF |

- Done as:

```
serial_sendSCIC(&SerialC, "L0\n", 3); // Turn LiDAR off
serial_sendSCIC(&SerialC, "L1\n", 3); // Turn LiDAR on
```

# LiDAR – Communication Settings

The default baud rate of the sensor through SCI is 19.2 kbps. Though it can be changed to 57.6, 115.2, 250, 500 and 750 kbps by varying the settings.

Example :

- 57.6 Kpbs = “057600” (ASCII 6 Digits)
- 115.2 Kpbs = “115200” (ASCII 6 Digits)

(HOST->SENSOR)

| | | | |
|---|---|---|---|
| ‘S’($53_{16}$) | Baud Rate (6 Digits) | Reserved Area (7 Digits) | LF or CR |

(SENSOR->HOST)

| | | | | | |
|---|---|---|---|---|---|
| ‘S’($53_{16}$) | Baud Rate (6 Digits) | Reserved Area (7 Digits) | Status | LF | LF |

What baud rate is being set here?

```
char S_command[19] = "S1152000124000\n";
uint16_t S_len = 19;
serial_sendSCIC(&SerialC, S_command, S_len);
```

The baud rate set here is, 115200

# LiDAR – Distance Data Acquisition

- The maximum measurable distance of the sensor is 4095 mm with 1 mm resolution.
- Each data point is represented with 12 bits (0-4095 range).
- To represents the data points in ASCII format a simple encoding is implemented.
- The 12-bit data is separated into 6 bit (max value of 63) each and then have $30_{16}$ (‘0’) added to them.

Example:

$$1234\ \text{mm} = 010011010010_2$$

$$\downarrow \text{separation}$$

$$(010011_2, 010010_2) = (13_{16}, 12_{16})$$

$$\downarrow \text{Add } 30_{16}$$

$$(43_{16}, 42_{16}) = (\text{‘C’}, \text{‘D’})$$

To decode you just do the inverse, where you subtract $30_{16}$ and then merge using bitwise or and bit shifting. It follows the big-endian system. The most significant bits of the data are sent first (in this case ‘C’ then ‘D’).

# LiDAR – Distance Data Acquisition

- The full range of the sensor is 240 degrees where it can send a maximum of 683. However, it has 768 configurable steps. To be able to specify the actual usable range that we want to observe we can set this in the ‘G’ command.

- To obtain the data, we need to assign the starting point, end point and cluster count.
- The sensor groups multiple neighboring points assigned by the cluster count.
- The minimum value from each group is supplied as distance data to the host

(HOST->SENSOR)

| | | | |
|---|---|---|---|
| ‘G’($47_{16}$) | Starting Point (3 Digits) | End Point (3 Digits) | Cluster Count (2 Digits) |
| LF or CR | | | |

- **Starting point** (0-768): Point of the area from where the data reading starts.
- **End Point** (0-768): Point of the area where the data reading stops.
- **Cluster Count** (0-99): Number of neighboring points that are grouped as a cluster.

# LiDAR – Distance Data Acquisition

(HOST->SENSOR)

| | | | |
|---|---|---|---|
| 'G'($47_{16}$) | Starting Point (3 Digits) | End Point (3 Digits) | Cluster Count (2 Digits) |
| LF or CR | | | |

```
char G_command[] = "G0447250 3\n"; //command for getting distance -120 to 120
degree
uint16_t G_len = 11;
serial_sendSCIC(&SerialC, G_command, G_len);
```

What is the starting point?

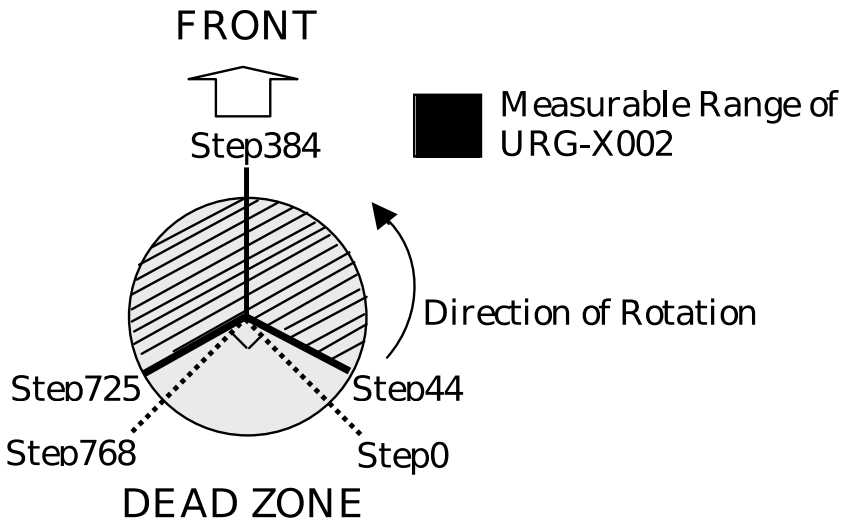
'044' = 44th step

What is the ending point?

'725' = 725th step

What is the cluster count?

'03' = group 3



# LiDAR – Distance Data Acquisition

- (HOST->SENSOR)

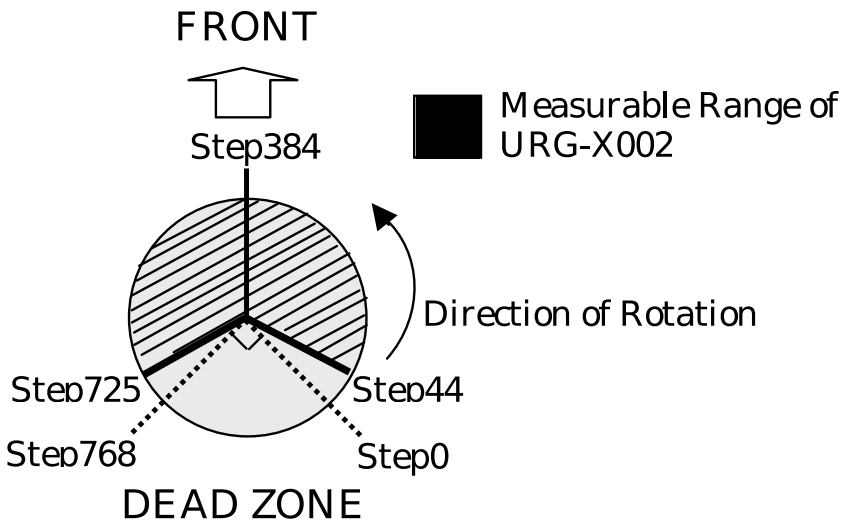
| | | | |
|---|---|---|---|
| 'G'($47_{16}$) | Starting Point (3 Digits) | End Point (3 Digits) | Cluster Count (2 Digits) |
| LF or CR | | | |

```
char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
uint16_t G_len = 11;
serial_sendSCIC(&SerialC, G_command, G_len);
```

How many datapoints are we expecting then based on the command?

$$\frac{\text{End step} - \text{start step}}{\text{Cluster count}} + 1$$

$$\frac{725 - 44}{3} + 1 = 228$$



# LiDAR – Distance Data Acquisition

- Sensor measures the distance in the range 20 mm to 4095 mm. If the distance is less than 20 mm, we can encode an error into the distance measurement.

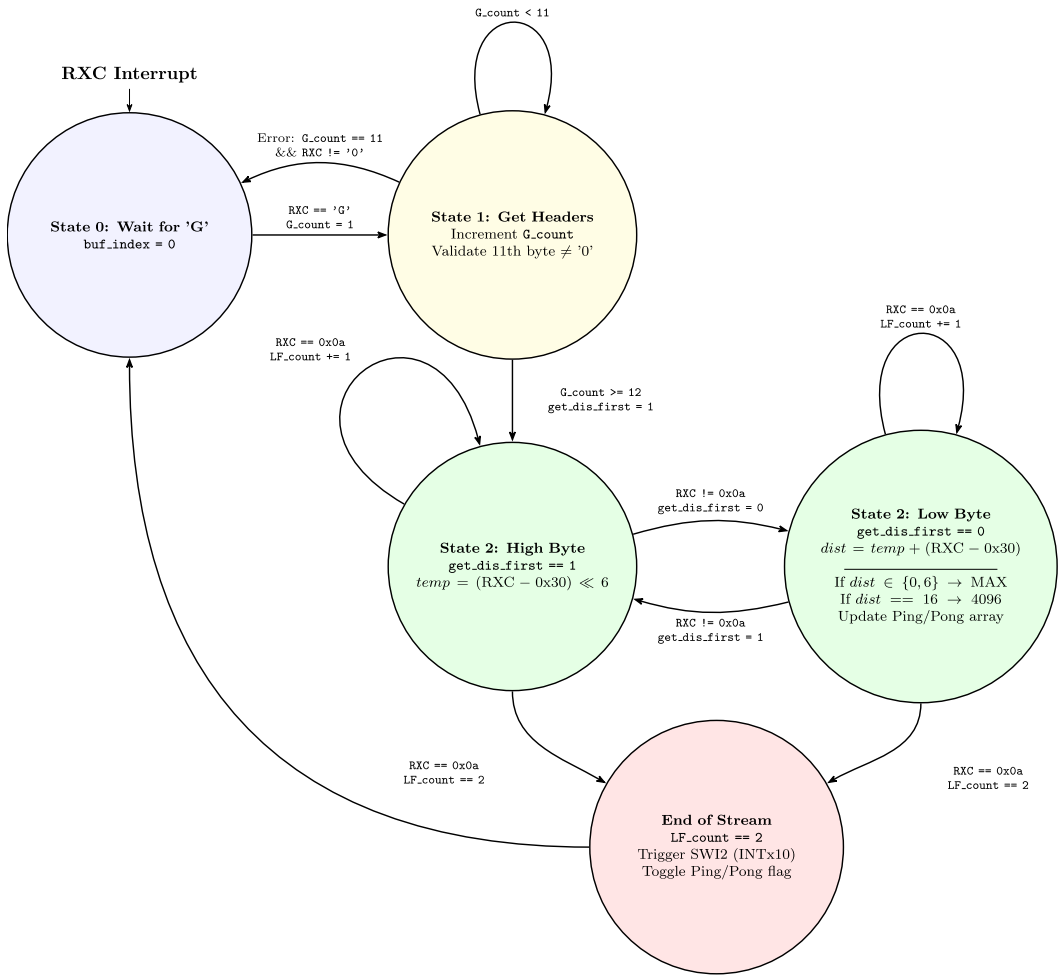
| Error Code | Error Type |
|---|---|
| 0 | Possibility of detected object is at 22m |
| 1 | Reflected light has low intensity |
| 2 | Reflected light has low intensity |
| 3 | Reflected light has low intensity |
| 4 | Reflected light has low intensity |
| 5 | Reflected light has low intensity |
| 6 | Possibility of detected object is at 5.7m |
| 7 | Distance data on the preceding and succeeding steps have errors |
| 8 | Others |
| 9 | The same step had error in the last two scan |
| 10 | Others |
| 11 | Others |
| 12 | Others |
| 13 | Others |
| 14 | Others |
| 15 | Others |
| 16 | Possibility of detected object is in the range 4096mm |
| 17 | Others |
| 18 | Unspecified |
| 19 | Non-Measurable Distance |

# LiDAR – Distance Data Acquisition

- We implement the reading of the data as a state machine,

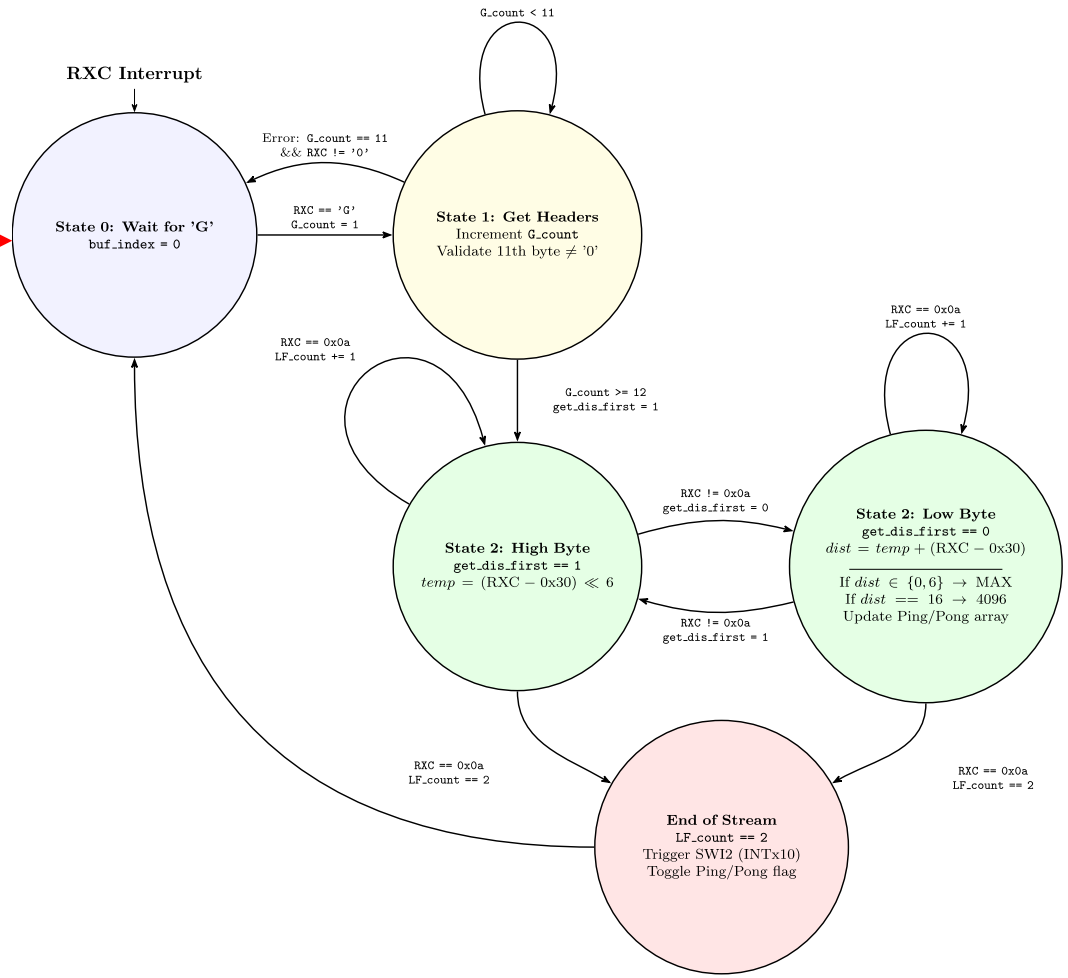
(SENSOR->HOST)

| | | | | |
|---|---|---|---|---|
| 'G'($47_{16}$) | Starting Point (3 Digits) | End Point (3 Digits) | Cluster Count (2 Digits) | LF |
| Status | | | | LF |
| Data Block 1 | | | | LF |
| ..... | | | | LF |
| Data Block N-1 (64 Byte) | | | | LF |
| Data Block N (n Byte) | | | | LF LF |



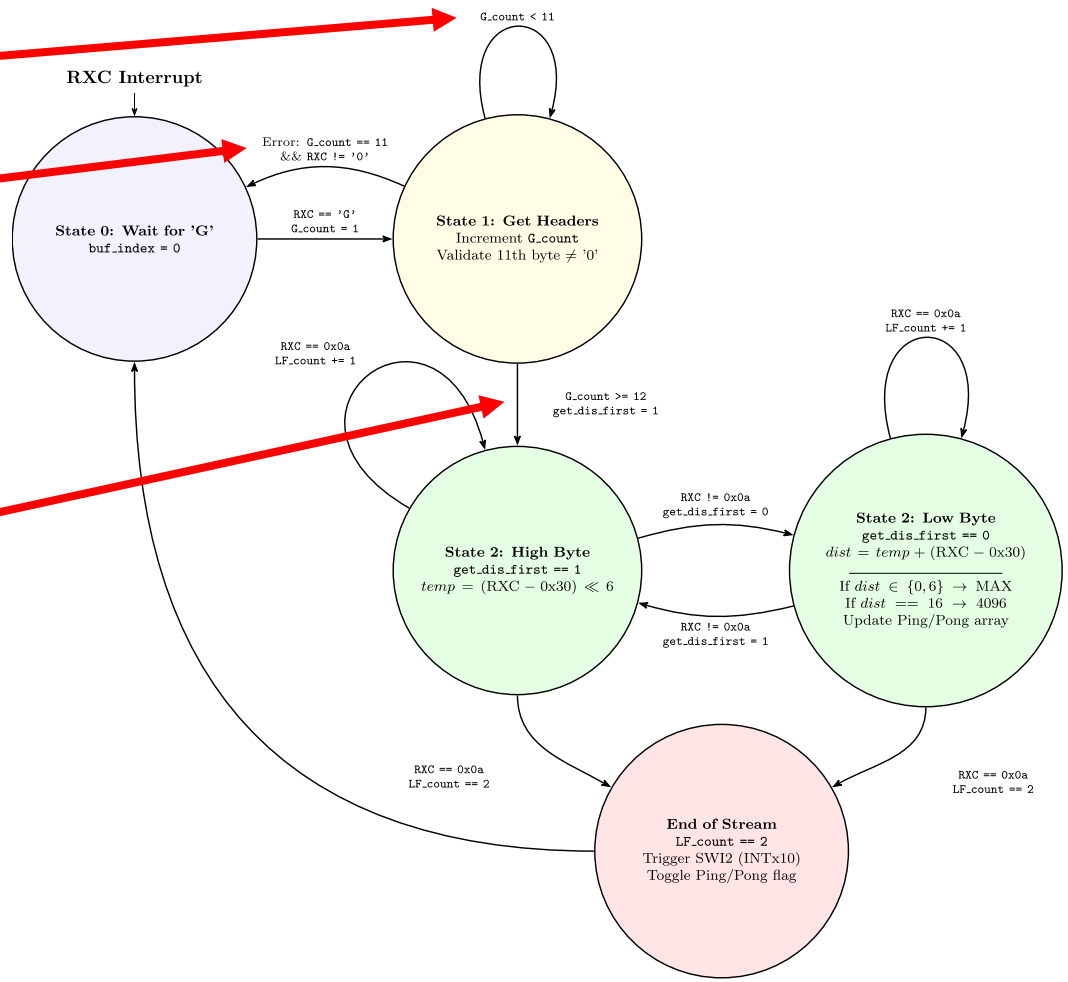
# LiDAR – Distance Data Acquisition

```
if (read_ladar_state == 0) {//wait for the
starting char 'G'
    buf_index = 0;
    if (RXCdata =='G') {
        read_ladar_state = 1;
        G_count = 1;
    }
}
```



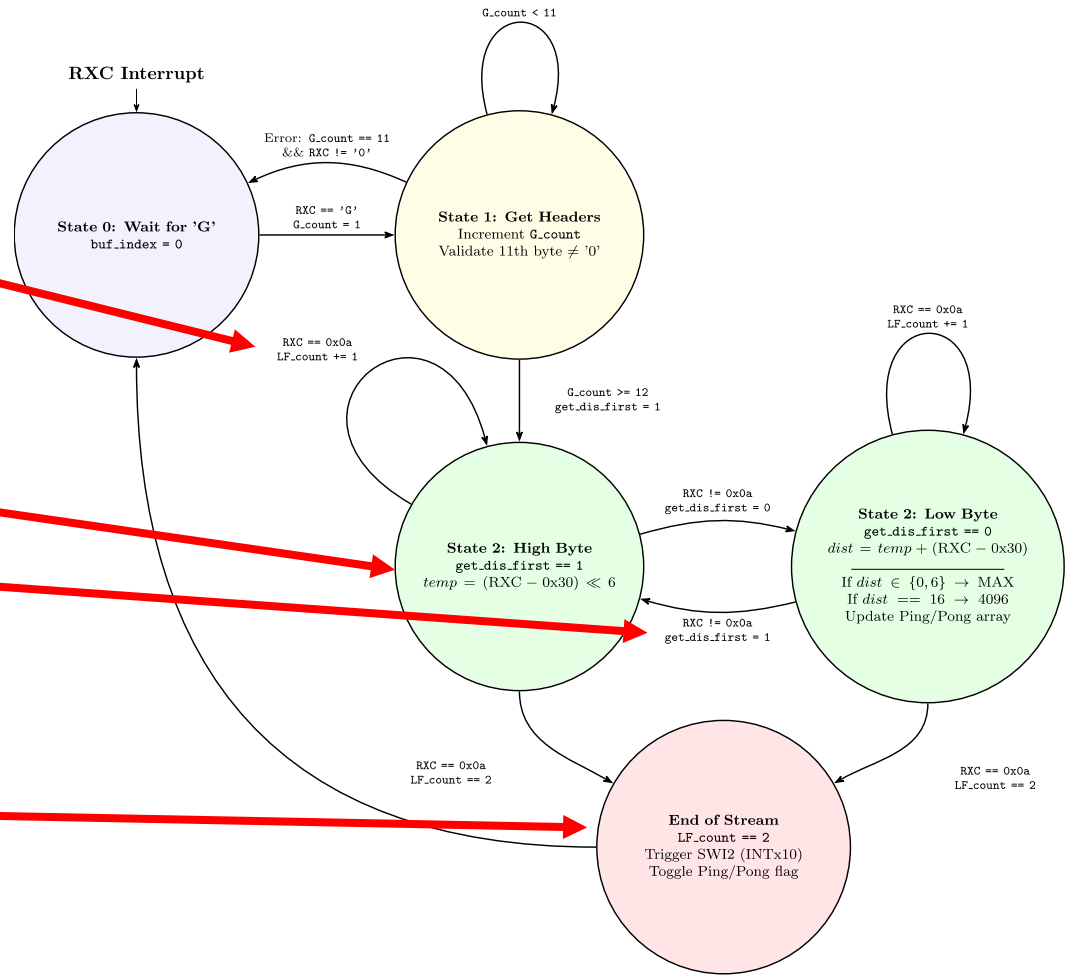
# LiDAR – Distance Data Acquisition

```
else if (read_ladar_state == 1) {//get the headers
    G_count += 1;//count the number of headers
    //check the status code and if it is not '0', wait for
the next packet
    if (G_count == 11) {
        if (RXCdata != '0') {
            read_ladar_state = 0;
        }
    }
    if (G_count >= 12) {//there are total 12 things before
the data
        read_ladar_state = 2;
        get_dis_first = 1;
        dis_index = 0;
    }
}
```



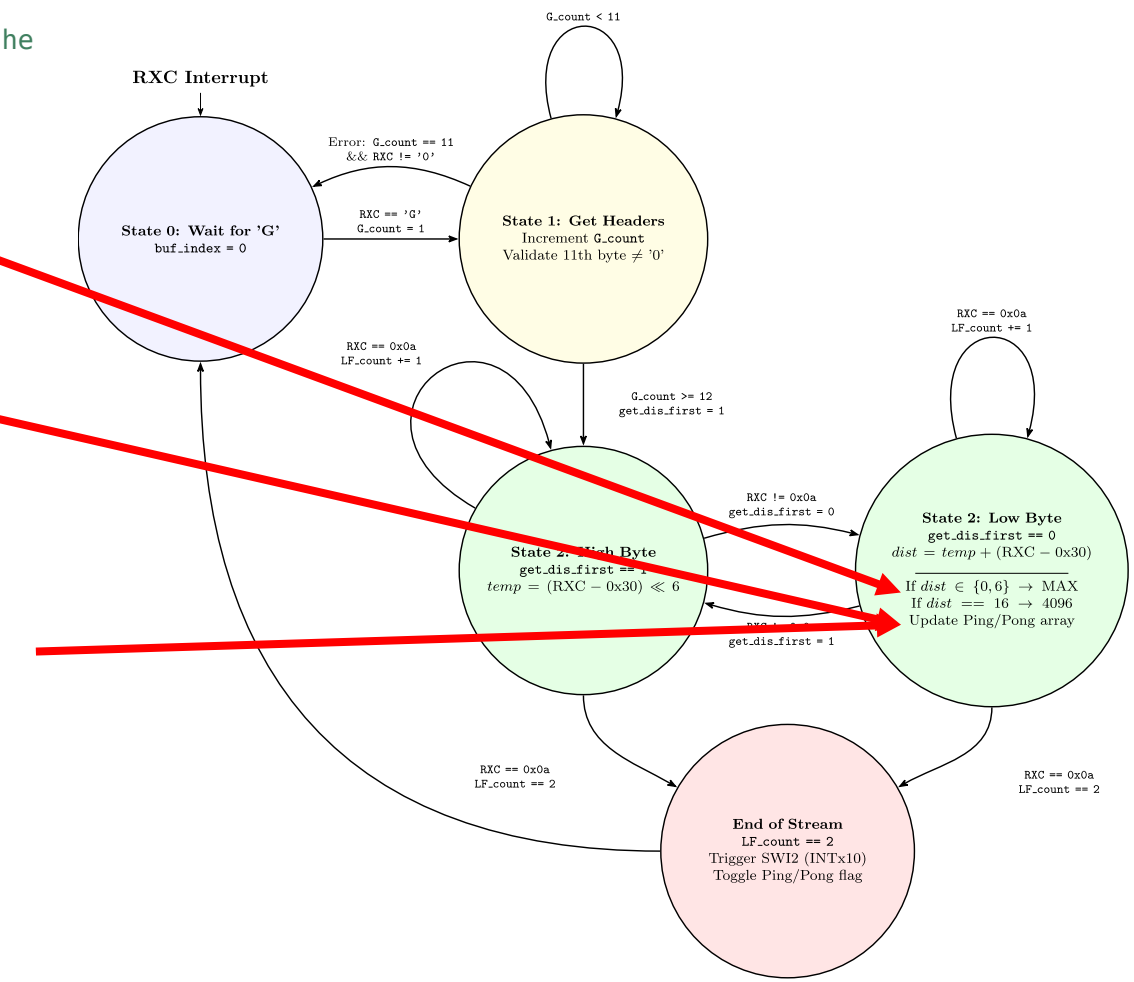
# LiDAR – Distance Data Acquisition

```
else if (read_ladar_state == 2) {//get actual data
    if (RXCdata == 0x0a) { //check for line feed
        LF_count += 1;
    } else {
        LF_count = 0;
        if (get_dis_first == 1) {
            tempscid = (RXCdata - 0x30) << 6;//first data is the top 6 bit of the distance
reading
            get_dis_first = 0;
        } else if (get_dis_first == 0) {
            // Continued to next slide
        }
    if (LF_count == 2) { //two consecutive line feed means the end of the current data stream
        read_ladar_state = 0;
        print_dis_index = dis_index;
        PieCtrlRegs.PIEIFR12.bit.INTx10 = 1; // Manually cause the interrupt for the SWI2
        LADARpingpong = 1-LADARpingpong;
    }
}
```



# LiDAR – Distance Data Acquisition

```
else if (get_dis_first == 0) {
    distance = tempscid + (RXCdata - 0x30);//second data is the bottom 6 bit of the
distance reading
    if (distance == 0 || distance == 6) {//error code
        distance = LADAR_MAX_READING; //set to the max reading
    } else if (distance == 16) { //distance is 4096mm
        distance = 4096;
    }
    if (LADARpingpong == 0) {
        checkdist = distance*0.00328084;
        if ((checkdist > 0.0) && (checkdist < 20.0)) {
            ladar_data[dis_index].distance_ping = checkdist; // fill in the
distance_ping array
        } else {
            numRCXerrors++;
            ladar_data[dis_index].distance_ping =
ladar_data[dis_index].distance_pong;
        }
    } else if (LADARpingpong == 1) {
        checkdist = distance*0.00328084;
        if ((checkdist >= 0.0) && (checkdist < 20.0)) {
            ladar_data[dis_index].distance_pong = checkdist; // fill in the
distance_ping array
        } else {
            numRCXerrors++;
            ladar_data[dis_index].distance_pong =
ladar_data[dis_index].distance_ping;
        }
    }
    dis_index += 1;
    get_dis_first = 1;
}
```



# LiDAR – Distance Data Acquisition

- The sensor has a scanning speed of 100 msec / scan ( so 10 Hz ) and we want to communicate with at a baud rate of 115200.
- As such in the main function we do the initialization:

```
ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // 10 Hz timer
init_serialSCIC(&SerialC,19200); //remember that the initial baud rate is 19200!

 char S_command[19] = "S1152000124000\n"; // this changes the baud rate to 115200
 uint16_t S_len = 19;
 serial_sendSCIC(&SerialC, S_command, S_len);


 DELAY_US(1000000); // Delay letting LADAR change its Baud rate
 init_serialSCIC(&SerialC,115200); // Use the new baud rate for communication


 for (LADARi = 0; LADARi < 228; LADARi++) { // the angle each point will be at will be constant
     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication
is faster; 0.3515625 is 360/1024
 }
```

# LiDAR – Distance Data Acquisition

- We use CPU Timer 1 to call the SCI command for reading the LiDAR information

```
__interrupt void cpu_timer1_isr(void)
{
    serial_sendSCIC(&SerialC, G_command, G_len);
    CpuTimer1.InterruptCount++;
}
```

So, every 10Hz timer 1 sends the ‘G’ command to collect the new data into the lidar_data buffer.

# LiDAR – Distance Data Acquisition

- Once we get all the information from the SCI interrupt it triggers a software interrupt

```
if (LF_count == 2) { //two consecutive line feed means the end of the current data stream
    read_ladar_state = 0;
    print_dis_index = dis_index;
    PieCtrlRegs.PIEIFR12.bit.INTx10 = 1; // Manually cause the interrupt for the SWI2
    LADARpingpong = 1-LADARpingpong;
}
```

- As a reminder a software interrupt is the lowest priority level and can be interrupted by any of the other interrupts of higher priority levels.
- The rows represent a **group**
- Each column a **channel** within that group
- When multiple interrupts are pending, the lowest-numbered channel from the lowest-numbered group is serviced first. Thus, the interrupts at the top left of the table are service first and the bottom right last.

# LiDAR – Distance Data Acquisition

**Table 3-2. PIE Channel Mapping**

| | INTx.1 | INTx.2 | INTx.3 | INTx.4 | INTx.5 | INTx.6 | INTx.7 | INTx.8 | INTx.9 | INTx.10 | INTx.11 | INTx.12 | INTx.13 | INTx.14 | INTx.15 | INTx.16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| INT1.y | ADCA1 | ADCB1 | ADCC1 | XINT1 | XINT2 | ADCD1 | TIMER0 | WAKE | - | - | - | - | IPC0 | IPC1 | IPC2 | IPC3 |
| INT2.y | EPWM1_TZ | EPWM2_TZ | EPWM3_TZ | EPWM4_TZ | EPWM5_TZ | EPWM6_TZ | EPWM7_TZ | EPWM8_TZ | EPWM9_TZ | EPWM10_TZ | EPWM11_TZ | EPWM12_TZ | - | - | - | - |
| INT3.y | EPWM1 | EPWM2 | EPWM3 | EPWM4 | EPWM5 | EPWM6 | EPWM7 | EPWM8 | EPWM9 | EPWM10 | EPWM11 | EPWM12 | - | - | - | - |
| INT4.y | ECAP1 | ECAP2 | ECAP3 | ECAP4 | ECAP5 | ECAP6 | - | - | - | - | - | - | - | - | - | - |
| INT5.y | EQEP1 | EQEP2 | EQEP3 | - | CLB1 | CLB2 | CLB3 | CLB4 | SD1 | SD2 | - | - | - | - | - | - |
| INT6.y | SPIA_RX | SPIA_TX | SPIB_RX | SPIB_TX | MCBSPA_RX | MCBSPA_TX | MCBSPB_RX | MCBSPB_TX | SPIC_RX | SPIC_TX | - | - | - | - | - | - |
| INT7.y | DMA_CH1 | DMA_CH2 | DMA_CH3 | DMA_CH4 | DMA_CH5 | DMA_CH6 | - | - | - | - | - | - | - | - | - | - |
| INT8.y | I2CA | I2CA_FIFO | I2CB | I2CB_FIFO | SCIC_RX | SCIC_TX | SCID_RX | SCID_TX | - | - | - | - | - | - | UPPA (CPU1 only) | - |
| INT9.y | SCIA_RX | SCIA_TX | SCIB_RX | SCIB_TX | CANA_0 | CANA_1 | CANB_0 | CANB_1 | - | - | - | - | - | - | USBA (CPU1 only) | - |
| INT10.y | ADCA_EVT | ADCA2 | ADCA3 | ADCA4 | ADCB_EVT | ADCB2 | ADCB3 | ADCB4 | ADCC_EVT | ADCC2 | ADCC3 | ADCC4 | ADCD_EVT | ADCD2 | ADCD3 | ADCD4 |
| INT11.y | CLA1_1 | CLA1_2 | CLA1_3 | CLA1_4 | CLA1_5 | CLA1_6 | CLA1_7 | CLA1_8 | - | - | - | - | - | - | - | - |
| INT12.y | XINT3 | XINT4 | XINT5 | - | - | VCU | FPU_OVERFLOW | FPU_UNDERFLOW | EMIF_ERROR | RAM_CORRECTABLE_ERROR | FLASH_CORRECTABLE_ERROR | RAM_ACCESS_VIOLATION | SYS_PLL_SLIP | AUX_PLL_SLIP | CLA OVERFLOW | CLA UNDERFLOW |

Note: Cells marked "-" are Reserved

# LiDAR – Distance Data Acquisition

In the LiDAR lab code, we will have 3 software interrupts:

Software interrupt 1, will have the **highest** priority, this is used for retrieving the data and sending the data to and from the RaspberryPI

Software interrupt 2, will have the **middle** priority, where we will perform the LiDAR position computation and the coordinate transformations based on the sensor's location on the robot.

Software interrupt 3, will have the **lowest** priority, where anything else can be done

In this lecture we will only look at software interrupt 2.

# LiDAR – Distance Data Acquisition

```
__interrupt void SWI2_MiddlePriority(void)     // RAM_CORRECTABLE_ERROR
{
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER &= MINT12;                          // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_10;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;       // Enable PIE interrupts so that other interrupts can go in between
    __asm("  NOP");
    EINT;

    if (LADARpingpong == 1) {
        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
        for (LADARi = 52, LADARrightfront = 19; LADARi <= 56 ; LADARi++) { // 19 is greater than max feet
            if (ladar_data[LADARi].distance_ping < LADARrightfront) {
                LADARrightfront = ladar_data[LADARi].distance_ping;
            }
        }
        // LADARfront is the min of dist 111, 112, 113, 114, 115
        for (LADARi = 111, LADARfront = 19; LADARi <= 115 ; LADARi++) {
            if (ladar_data[LADARi].distance_ping < LADARfront) {
                LADARfront = ladar_data[LADARi].distance_ping;
            }
        }

        // Do coordinate transformations here, done in the next slides

    } else if (LADARpingpong == 0) { /** same for pong */ }
}
```

# LiDAR – Distance Data Acquisition

- We start with the coordinate transform to get the coordinate of the LiDAR in the real world based on the robot's position and heading.

$$\begin{bmatrix} x_W \\ y_W \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_R \\ y_R \\ 1 \end{bmatrix}$$

```
LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
```

Now we can get the coordinates of the object in the global reference frame based on the distance returned from the LiDAR and the angle.

```
for (LADARi = 0; LADARi < 228; LADARi++) {
    ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
    ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
}
```

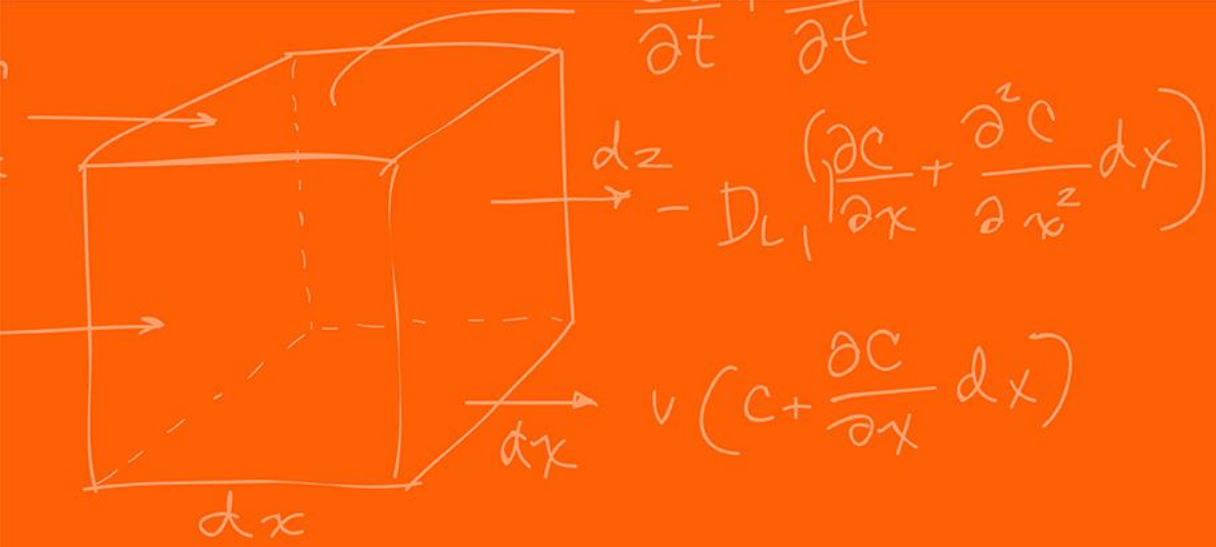
# LiDAR – Distance Data Acquisition

The pose objects as simply defined as,

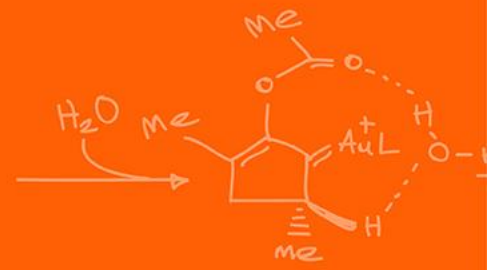
```
typedef struct
{
    float x;        //in feet
    float y;        //in feet
    float theta;    // in radians between -PI and PI. 0 radian is along the +x axis, PI/2
is the +y axis
} pose;

pose ROBOTps = {0,0,0}; //robot position, will be updated through your integration code
pose LADARps = {3.5/12.0,0,1};  // 3.5/12 for front mounting, theta is not used in this
current code
```

# Questions?









# The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

